

Application Security

DevSecOps - KodeCloud

[Talisman](#) – Keep your secrets secret

[PITest](#) – Mutation Tests

[SonarQube](#) - SAST

[Dependency-Check](#) Check your dependency

[Trivy](#) - Scan your Images

[OPA Conftest](#) – Docker Security

[OWASP ZAP](#) – DAST

OWASP TOP 10

1. Broken Access Control

- Practice
 - [Path traversal](#)
 - [Information disclosure vulnerabilities](#)
 - [Access control vulnerabilities and privilege escalation](#)
 - [File upload vulnerabilities](#)
- **Description:** Failure to restrict what authenticated users are allowed to do. Failures occur when the application does not enforce policy, allowing users to access data or functions they are not authorized to use.
- **Vulnerability Types:**
 - **IDOR (Insecure Direct Object Reference):** Directly accessing objects (like user profiles or records) by changing identifiers in the URL or API request.
 - **Privilege Escalation:** A standard user successfully changing their role to "Admin" by manipulating a request.
 - **Missing Function/Object Level Authorization:** API endpoints that lack proper checks, allowing unauthorized users to execute sensitive tasks.
 - **Path Traversal:** Accessing restricted server files by manipulating directory paths.
- **Real-World Case Study (The Optus Breach, 2022):**
 - A major telecommunications provider suffered a breach exposing 10 million records.
 - The failure was two-fold: First, an API endpoint was completely unprotected. Second, the system used **sequential IDs** rather than high-entropy (random) strings.
 - Combined, these allowed an attacker to enumerate through the records by simply incrementing the ID by one.
- **Solutions:**
 - **Deny by Default:** Access to data and functions should be the exception, not the rule.
 - **High Entropy IDs:** Use random, non-sequential strings for identifiers to prevent enumeration.
 - **Rate Limiting:** Implement strict rate limiting on API endpoints to stop brute-force or enumeration attempts.
 - **Tools:** Utilize **DAST** (like OWASP ZAP) to simulate attacks, **SAST** to scan code for patterns, and **In-App Firewalls** to block malicious requests in real-time.

- **Port Swagger Notes**
 - **Vertical Access Control (Who are you?)**
 - **The Idea:** Administrators have high-level power; regular users have limited power.
 - **Example:** Only a manager can delete user accounts, while an employee can only view them.
 - **Horizontal Access Control (Which items belong to you?)**
 - **The Idea:** You are restricted to your own "lane."
 - **Example:** You can view your own bank account transactions, but you cannot see your neighbor's account.
 - **Context-Dependent Access Control (Is it the right time/order?)**
 - **The Idea:** You must follow the correct process.
 - **Example:** Once you have paid for your order in a shopping cart, the system locks that cart so you cannot go back and add or remove items.
- **Finding Vulnerability (Labs)**
 - **Vertical**
 - **Unprotected admin functionality**
 - /Robots.txt
 - **Unprotected admin functionality with unpredictable URL**
 - Source Code
 - **Parameter-based access control methods**
 - Request Data (Token, Cookie, Params)
 - **URL-based access control can be circumvented**
 - X-Original-Url: Header
 - **Method-based access control can be circumvented**
 - Change Request Method
 - **Case Sensitive Access Control**
 - When ADMIN maps to admin
 - Access Control only restrict admin

- **Horizontal (IDOR)**
 - UserId Controlled by request parameter
 - User ID controlled by request parameter, with unpredictable user IDs
 - User ID controlled by request parameter with data leakage in redirect
 - User ID controlled by request parameter with password disclosure
 -
 -

2. Security Misconfiguration

- **Description:** Unlike code-level vulnerabilities, this category focuses on the insecure hosting, environment, or configuration of an application. It occurs when security controls are applied improperly, left at default settings, or exposed unnecessarily.
- **Vulnerability Types:**
 - **Exposed Cloud Storage:** Publicly accessible storage buckets (e.g., S3, Azure Blobs) containing sensitive data.
 - **Default Credentials:** Using factory-set passwords (e.g., `admin:admin`) for administrative dashboards.
 - **Excessive Verbosity:** Exposing system details, such as full stack traces or debug info, in error messages.
 - **Unnecessary Services/Ports:** Keeping non-essential ports (like SSH port 22) open and exposed to the internet.
 - **Directory Exposure:** Publicly accessible sensitive files like `.git` directories, which reveal the entire source code.
- **Real-World Case Studies:**
 - **Verizon Breach (2017):** A publicly accessible S3 bucket exposed sensitive customer data, including phone numbers, account PINs, and service details, creating a high risk for spear-phishing and SIM-swapping.
 - **NVIDIA (Lapsus\$ Group):** The hacking group used credential stuffing campaigns against admin panels to break in and steal sensitive records.
 - **SSH/Service Exploits:** Various breaches, including attacks on Tesla's Kubernetes cluster, malware deployment on European supercomputers (CalaS), and mass cryptomining infections (XM rig), were all facilitated by exposed SSH ports or poor credential management.
- **Prevention & Mitigation:**
 - **CSPM (Cloud Security Posture Management):** Implement CSPM tools that act as "insider experts" to continuously scan cloud accounts, identify misconfigured assets (like open ports or public buckets), and suggest fixes.
 - **Asset Management:** You cannot secure what you don't know exists. Use automated tools to discover all cloud assets.
 - **Hardening SSH:** Never use root logins. Disable password authentication in favor of SSH keys.
 - **Automated Tooling:** Use scanners to check for common misconfigurations during the build process, and ensure that example applications and default configs are removed from production environments.

3. Software Supply Chain Failures

- **Description:** Trusting unvetted third-party code or tools, compromising the build.
- **Vulnerability Types:**
 - **Dependency Poisoning:** Malicious packages in NPM/PyPI/etc.
 - **Typosquatting:** Downloading a fake package with a similar name.
 - **Compromised CI/CD:** Malicious code injected during the build process.
 - **Malicious IDE Extensions:** Compromised plugins (VS Code/Cursor).
- **Examples:** A popular open-source package being updated to include a backdoor.
- **Solutions:** Software Composition Analysis (SCA); SBOM (Software Bill of Materials).

4. Cryptographic Failures

- **Description:** Inadequate protection of data at rest or in transit, often caused by weak algorithms or poor key management.
- **Vulnerability Types:**
 - **Weak/Deprecated Algorithms:** Using outdated hashing or encryption protocols that have known vulnerabilities.
 - **Improper Key Management:** Hardcoding keys in source code or storing secrets in insecure locations (e.g., Git repositories).
 - **Lack of Encryption:** Storing sensitive data in plain text or failing to encrypt data during transit.
 - **Improper Protocol Implementation:** Using weak transport protocols like SSL or older versions of TLS.
- **Real-World Case Studies:**
 - **Freecycle Breach:** The company suffered a significant breach involving usernames and emails. While the passwords were technically hashed, the use of the outdated **MD5** algorithm rendered the protection effectively useless, as the hashes were easily crackable.
 - **Paris Lockdown App:** An application designed to manage movement during the 2020 COVID-19 lockdown correctly encrypted user data but **hardcoded the encryption key** directly in the source code alongside the encryption logic, rendering the encryption effectively pointless.
- **Solutions:**
 - **Use Modern Standards:** Never use MD5, SHA1, or DES for security-sensitive operations. Avoid TLS 1.2 or lower, and never use SSL 2.0 or 3.0.
 - **Secure Key Management:** Never hardcode passwords or encryption keys in source code or version control. Use dedicated **Secrets Managers** to handle credentials.
 - **Encrypt Everything Sensitive:** Ensure passwords are never stored in plain text and use robust hashing algorithms (e.g., Argon2, bcrypt).
 - **Automated Tooling:**
 - **SAST:** Scans source code to identify usage of outdated cryptographic algorithms.
 - **DAST:** Attacks the application from the outside to identify exposed security flaws.
 - **Secrets Detection:** Scans code and commit history to ensure no API keys or hardcoded passwords are leaked.
 - **Cloud Management:** Ensures assets like data buckets are configured with appropriate encryption and permissions.

5. Injection

- **Description:** The process of inserting malicious code into untrusted input fields, forcing the application to execute that code to steal data, bypass authentication, or disrupt services.
- **Vulnerability Types:**
 - **SQL Injection (SQLi):** Manipulating database queries by injecting SQL commands (e.g., ' OR '1'='1') to bypass login screens or extract unauthorized data.
 - **Code Injection:** Injecting language-specific commands (e.g., JavaScript `process.kill`) into input fields to crash the server or execute unauthorized system operations.
 - **Cross-Site Scripting (XSS):** Injecting malicious scripts (typically JavaScript) into a web page that get executed in the browsers of other users, often to steal session cookies or tokens.
 - **NoSQL/LDAP Injection:** Targeting non-relational or directory databases with malicious query payloads.
- **Key Concepts:**
 - **Input Handling:** All injection attacks share the same flaw: treating user-provided, untrusted input as executable code rather than plain text.
 - **Automation:** Injection vulnerabilities are highly susceptible to automated exploitation, as attackers can use lists of "payloads" to scan hundreds of sites simultaneously (as seen in the "Resume Looters" campaign).
 - **Session Theft (XSS):** By stealing session cookies via XSS, attackers can impersonate users and bypass Multi-Factor Authentication (MFA) because the system treats the stolen token as a "trusted" session.
- **Real-World Breaches:**
 - **MOVEit Breach (2003):** A significant SQL injection attack that caused widespread disruption, including the temporary grounding of some airlines.
 - **Resume Looters:** An automated, large-scale campaign that targeted hundreds of sites, successfully compromising 65 websites to harvest sensitive personal data.
- **Solutions:**
 - **Prepared Statements (SQLi Specific):** Always use parameterized queries where the query structure is defined first, and data is treated strictly as a parameter.
 - **Input Validation:** Use "allow-listing" (defining what characters are permitted) rather than "block-listing" (trying to guess all bad characters).

- **Escaping:** If you must allow special characters, convert them into safe strings (e.g., using backslashes or HTML entities) so the server treats them as text, not code.
- **In-App Firewalls:** Use tools that understand application context to intercept and block malicious patterns between the user and the database (e.g., Zen by Aikido).
- **Automated Tooling:** Use **SAST** to scan code for injection patterns, **DAST** to perform automated penetration tests against the live app, and secrets detection to ensure no data is inadvertently exposed.

6. Insecure Design

Description: Vulnerabilities stemming from fundamental flaws in the application's architecture or logic, rather than implementation errors. While other vulnerabilities (like Injection) are about *how* code is written, Insecure Design is about *what* was designed in the first place.

Vulnerability Types:

- **Business Logic Flaws:** Manipulating workflows to achieve unintended outcomes (e.g., negative money transfers, logic errors in exchange rates).
- **Overexposed Attack Surface:** Leaving internal administrative tools or legacy systems accessible to the public internet.
- **Insecure Error Handling:** Exposing sensitive system information, stack traces, or backend details in user-facing error messages.
- **Insufficient Logging & Monitoring:** A design failure where an application is built without the capability to detect or alert on suspicious activities.

Real-World Case Study (Coinbase "Infinite Money" Loop):

- For years, a vulnerability existed in the Coinbase cryptocurrency exchange where the backend API validated that a user had enough *total assets* (by count) to cover a transaction, but it failed to distinguish between the *types* of assets (e.g., Bitcoin vs. Ethereum).
- An attacker could craft a request to send Bitcoin, and the system would incorrectly authorize it if the user possessed enough lower-value coins (Ethereum), effectively allowing the user to drain funds they did not actually own.

Prevention & Mitigation:

- **Threat Modeling:** Regularly map out your entire attack surface, including microservices and API endpoints.

S	Spoofing	Authentication	Pretending to be something or someone other than yourself
T	Tampering	Integrity	Modifying something on disk, network, memory, or elsewhere
R	Repudiation	Non-Repudiation	Claiming that you didn't do something or were not responsible.
I	Information Disclosure	Confidentiality	Providing information to someone not authorized to access it.
D	Denial of Service	Availability	Exhausting resources needed to provide a service.
E	Elevation of Privilege	Authorization	Allowing someone to do something they are not authorized to do.

- **STRIDE Framework:** Use this to analyze assets for **S**poofing, **T**ampering, **R**epudiation, **I**nformation Disclosure, **D**enial of Service, and **E**levation of Privilege.
-
- **MITRE ATT&CK Framework:** Use this to understand the reconnaissance and attack phases an adversary might follow, helping to identify and secure assets at every step.
- **Leverage Established Frameworks:** Build applications using well-vetted, industry-standard frameworks rather than "reinventing the wheel," as these often include built-in logic safeguards.
- **Security Tooling:** While SAST and DAST are excellent for finding code-level bugs, they often miss business logic flaws. Therefore, **manual threat modeling** and architectural reviews are essential for this category.

7. Authentication Failures

Description: Weaknesses in verifying a user's identity. While *Broken Access Control* (A01) deals with what a user can do *after* logging in, *Authentication Failures* focus on the login process itself—essentially tricking the application into believing an attacker is a legitimate user.

Vulnerability Types:

- **Credential Stuffing & Brute Force:** Attackers use automated tools to "stuff" lists of known passwords (from past breaches, like the "RockYou" list) into login fields, or brute-force random character combinations.
- **Default Credentials:** Using factory-set passwords (e.g., `admin:admin`) for applications or IoT devices.
- **Session Management Failures:** Mismanaging session IDs/tokens (e.g., failing to expire tokens or failing to rotate them after suspicious events), which can allow attackers to hijack sessions and bypass Multi-Factor Authentication (MFA).
- **Insecure Password Reset Flows:** Improper design of "forgot password" mechanisms, such as exposing old passwords or using guessable reset tokens.

Real-World Case Studies:

- **Disney+ Launch (2019):** Attackers utilized credential stuffing shortly after the platform's launch to take over hundreds of user accounts.
- **Mirai Botnet:** This attack targeted IoT devices (printers, smart light switches, etc.) by exploiting default credentials, amassing an "army" of compromised devices to launch large-scale distributed attacks.
- **Facebook "View As" Bug:** A session management flaw where a feature meant to show how a profile appears to others unintentionally generated a valid session token for that account, allowing attackers to hijack other users' sessions.

Prevention & Mitigation:

- **Strong Password Hashing:** Use modern, secure hashing algorithms like `bcrypt` or `Argon2`. Never use outdated methods like MD5.
- **Rate Limiting:** Implement strict rate-limiting on login attempts to block automated brute-force attacks.
- **Mandatory MFA:** Ensure Multi-Factor Authentication is available and, for sensitive accounts, enforce its use.

- **Secure Session Lifecycle:** Invalidate session tokens after a period of inactivity and regenerate them upon triggering events like logging in from a new device, a different IP, or during "suspicious" hours.
- **No Default Passwords:** Never ship applications or devices with default credentials. If an account must be created, force a password change upon the first login.
- **Secure Reset Flows:** Never reveal existing passwords during a reset; only allow users to create a new one.
-

8. Software & Data Integrity Failures

- **Description:** Trusting data/code updates without verifying the source.
- **Vulnerability Types:**
 - **Insecure Deserialization:** Executing code hidden in serialized objects.
 - **Unverified Updates:** Installing patches without verifying digital signatures.
 - **Unauthorized Configuration Changes:** Modifying files during runtime.
- **Examples:** Attacker modifies a signed binary to include malware.
- **Solutions:** Use digital signatures (GPG/Code Signing); verify hashes.

9. Logging & Alerting Failures

- **Description:** Insufficient monitoring preventing breach detection.
- **Vulnerability Types:**
 - **Incomplete Logging:** Not recording auth/input events.
 - **Lack of Alerting:** Failing to notify admins of detected attacks.
 - **Log Tampering:** Attackers deleting logs to hide footprints.
- **Examples:** A breach goes unnoticed for months due to no active monitoring.
- **Solutions:** Centralized logging; real-time monitoring/SIEM integration.

10. Mishandling Exceptional Conditions

- **Description:** Application instability under unexpected input.
- **Vulnerability Types:**
 - **Buffer Overflows:** Overloading memory with massive payloads.
 - **Unhandled Exceptions:** Errors revealing system/backend details.
 - **Resource Exhaustion (DoS):** Triggering complex logic to crash the CPU/Memory.
- **Examples:** Sending special Unicode characters that cause a database crash.
- **Solutions:** Robust error handling (`try-catch`); Fuzzing.